Binary Search on Answer

```
class Solution {
    public int splitArray(int[] nums, int k) {
        int start = 0;
        int end =0;

        for(int i=0; i< nums.length; i++){
            start = Math.max(start, nums[i]);
            end += nums[i];
        }

        //Binary Search
        while(start < end){
            int mid = start+ (end - start)/2;

            int sum =0;
            int pieces = 1;
            for(int num: nums){
                if(sum + num > mid){
                    sum = num;
                    pieces++;
                }
                else{
                    sum += num;
                }
            }

            if(pieces <= k){
                end = mid;
            }
            else{
                start = mid+1;
            }
        }
        return start;


    }
}
```

This is a **very important pattern** (Binary Search on Answer), so let's break it down simply.

🔍 What problem is this solving?

You're solving:

👉 Split array into **k subarrays** such that
👉 the **maximum sum of any subarray is minimized**

🧠 Core Idea (Simple)

Instead of deciding *where to split*, you are:

➡️ Guessing the **answer (max subarray sum)**
➡️ Then checking: *"Can I split into ≤ k parts with this max sum?"*

That's why Binary Search works.

📌 Step 1: Search Space

```
int start = max element of array
int end = sum of entire array
```

Why?

- Minimum possible answer = largest element (you can't split a number)
- Maximum possible answer = whole array (no split)

🔁 Step 2: Binary Search

```
while(start < end)
```

We try a middle value:

```
int mid = start + (end - start)/2;
```

👉 Assume: **max allowed subarray sum = mid**

⚙️ Step 3: Check feasibility

This part is the heart:

```
int sum = 0;
int pieces = 1;

for(int num : nums){
    if(sum + num > mid){
        sum = num;
        pieces++;
    } else {
        sum += num;
    }
}
```

What is happening?

- Keep adding elements to current subarray

- If it exceeds **`mid`**, start a new subarray

👉 So **`pieces`** = number of subarrays needed

🎯 Step 4: Decision

```
if(pieces <= k){
    end = mid;      // try smaller max sum
} else {
    start = mid + 1; // need bigger sum
}
```

Why?

- If we can split in ≤ k parts → we can try smaller answer
- If we need more than k parts → mid is too small

✅ Final Answer

```
return start;
```

Because at the end:

👉 **`start == end`** → smallest valid maximum sum

🧩 Intuition in One Line

👉 "Find the smallest number such that the array can be split into k parts without any part exceeding that number."

⚡ Example

`nums = [7,2,5,10,8], k = 2`

Possible splits:

- **`[7,2,5] | [10,8]`** → max = 18 ✅
- **`[7,2] | [5,10,8]`** → max = 23 ❌ worse

Answer = **18**

🚀 Pattern You Should Learn

This is called:

👉 **Binary Search on Answer**

Used in problems like:

- Allocate books
- Painter's partition
- Capacity to ship packages
- Split array largest sum

💡 How to Learn This (your earlier concern)

Don't memorize syntax like `sum += num` or `charAt()` blindly.

Instead:

1. Understand **pattern**
2. Practice 5–6 problems of same type
3. Your brain will auto-recognize it

If you want, I can give you:
✅ a dry run step-by-step
✅ similar problems to practice
✅ or convert this into a super simple template you can reuse

Practice 5–6 problems of same type
where to find these

Good question — this is exactly how you *actually learn patterns* like this.

I'll give you **specific places + exact problems (not random links)** so you don't get overwhelmed.

🎯 Best Places to Practice This Pattern

1. 🟠 LeetCode (BEST)

Search this:

👉 **"Binary Search on Answer"**

🔥 Must-do problems (start here)

1. **Split Array Largest Sum** (the one you solved)
2. **Capacity To Ship Packages Within D Days**
3. **Koko Eating Bananas**

4. **Allocate Books** *(premium but similar versions exist)*
5. **Minimum Limit of Balls in a Bag**
6. **Magnetic Force Between Two Balls**

👉 Do these 5–6 → pattern will click permanently

---

2. 🟢 GeeksforGeeks (Beginner-friendly explanations)

Search:

👉 **"Binary Search on Answer GFG"**

Good problems there:

- Allocate Minimum Number of Pages
- Painters Partition Problem
- Aggressive Cows

👉 These are classic versions of the same idea

---

3. 🔵 CodeStudio

Search:

👉 **"Binary Search on Answer problems"**

Good curated lists + structured learning

🧠 How YOU should practice (important)

Don't just solve randomly. Do this:

Step 1:

Solve **1 problem fully**

Step 2:

Ask yourself:

- What was the **search space?**
- What was the **condition check?**

Step 3:

Immediately solve another similar one

⚡ Shortcut to Master Faster

All these problems follow SAME structure:

```
1. Define start and end
2. Binary search mid
3. Check if mid is valid
4. Shrink search space
```

👉 Only the **check() function changes**

🚀 If you want I can help you further:

I can:
✅ Give you a **perfect order (easy → hard)**
✅ Walk through 1 problem fully step-by-step
✅ Give you a **template you can reuse in exams/interviews**

Just tell me 👍

I can: ✅ Give you a **perfect order (easy → hard)** ✅ Walk through 1 problem fully step-by-step ✅ Give you a **template you can reuse in exams/interviews**
yes

Perfect — let's do this properly so you **actually master the pattern**, not just solve one question.

🚀 1. Perfect Order (Easy → Hard)

Start in this exact sequence on LeetCode:

🟢 Beginner (understand idea)

1. **Koko Eating Bananas**
2. **Capacity To Ship Packages Within D Days**

👉 These are easiest to *see the pattern*

🟡 Medium (core mastery)

3. **Split Array Largest Sum** ✅ (you already did)
4. **Minimum Limit of Balls in a Bag**

🔴 Advanced (deep understanding)

5. **Magnetic Force Between Two Balls**
6. **Aggressive Cows** (on GeeksforGeeks)

🧠 2. Step-by-Step Walkthrough (Koko Eating Bananas)

Problem idea:

Koko eats bananas at speed `k`

👉 Find **minimum k** such that she finishes in `h` hours

🔍 Step 1: Search Space

```
start = 1
end = max(piles)
```

🔁 Step 2: Try a speed (mid)

```
mid = (start + end)/2
```

⚙️ Step 3: Check if valid

For each pile:

```
hours += ceil(pile / mid)
```

👉 total hours needed

🎯 Step 4: Decision

- If `hours <= h` → valid → try smaller speed
- Else → increase speed

💡 Key Insight

👉 Smaller speed = more hours
👉 Bigger speed = fewer hours

So it's **monotonic** → **Binary Search works**

🧩 3. Universal Template (IMPORTANT)

This is what you should remember 👇

```
int start = ...; // min possible answer
int end = ...;   // max possible answer

while(start < end){
    int mid = start + (end - start)/2;

    if(isValid(mid)){
        end = mid;       // try smaller answer
    } else {
        start = mid + 1; // need bigger answer
    }
}

return start;
```

🧪 The ONLY thing that changes:

```
boolean isValid(int mid){
    // depends on problem
}
```

🔥 4. Recognizing This Pattern in Exams

Whenever you see:

- “minimum of maximum”
- “maximize minimum”
- “split into k parts”
- “within D days”
- “minimum capacity / speed”

👉 **Immediately think: Binary Search on Answer**

⚡ 5. Your Next Step (Do This Now)

👉 Solve:

1. **Koko Eating Bananas**
2. Then come back and say: *“done”*

I’ll:

- check your approach
- fix mistakes
- level you up to next problem

If you follow this properly, this pattern will become **one of your strongest topics** 💯